

A Report On Halfmoon: Log-Optimal Fault-Tolerant Stateful Serverless Computing

Submitted by
Proteeti Prova Rawshan
Matrikel. Nr. 63752
DSOS Seminar, WS 2023/24
Research in Computer and Systems Engineering
Technische Universität Ilmenau

Supervisor: Prof. Dr. Boris Koldehofe

January 29, 2024

Abstract

This report serves as an in-depth literature review of the publication "Halfmoon: Log-Optimal Fault-Tolerant Stateful Serverless Computing". It proposes a serverless runtime system to address the critical challenges of fault tolerance in stateful serverless computing. Traditional methods rely heavily on symmetric logging for both read and write operations, which introduces a significant overhead by impacting the performance & scalability of serverless applications. Halfmoon emerges as a novel solution, proposing asymmetric logging protocols that selectively log either reads or writes, but not both at the same time. This approach significantly reduces logging overhead whilst ensuring exactly-once semantics, a crucial requirement for fault-tolerant operations in distributed systems.

1 Introduction

In recent years, there has been a significant shift towards serverless computing - particularly through the Function-as-a-Service (FaaS) model [1–3]. This evolution as well as the adaptability of serverless computing has reduced the burden of resource management amongst the developers. This paradigm shift where the compute & storage are separated, has led to innovations in application deployment & management. However, it has also introduced some significant challenges in the state management of functions & fault tolerance, especially in the case of Stateful Serverless Functions (SSFs).

To achieve fault tolerance, load balancing & elastic scaling of resources are needed. However, they do not guarantee that the local state of the function will persist across invocations. Hence for sharing states across multiple functions, external storage is being used for primary state management. As SSFs are decomposed into stateless functions & external states, fault tolerance needs to be achieved separately in these 2 components in order to achieve as a whole unit. Because during application-level failures, there might be inconsistency via retries, which will result in anomalies. For example, an SSF that writes to an external state crashes, retrying this will duplicate the write that has already been applied. The goal should be, no matter how many times the function gets crashed & gets re-executed, the output will be the same (idempotence) with exactly-once semantics [4–6]-which is regardless of failures or retries. It focuses on the end-to-end guarantee of one-time execution. It's also observed that log based fault tolerance is useful in achieving it, which is already being used by the state-of-the art solutions [7]. They also show that to achieve the exactly-once semantics for SSFs, logging is required for *both* reads and writes.

However, logging operations has its own costs. existing approaches Boki & Beldi both show that their reads & writes are more expensive, accounting for more overhead [6, 7]. In addition.

several *concurrent* function instances might correspond to the same SSF invocation. For example, if an instance times out but is still alive due to a network error, the runtime might assume it's crashed and launch another to achieve fault tolerance. As a consequence, possibility of two race conditions are there. Firstly, a re-executed SSF might race with previously failed invocation of the same SSF-which might lead to duplication of a completed step. Secondly, an SSF might race with peer instances.

All of these aforementioned issues motivate the novel solution Halfmoon, which proposes to use parameterized timestamps to find interdependencies between events. Then leveraging them, authors propose 2 protocols where it's either log-free or write-free, depending on the workload & halving the logging overhead. A criterion is also provided for choosing the right protocol & a pauseless switching between these two during dynamic workloads.

2 Related Work

In the serverless computing paradigm, state management remains a key challenge & there has been significant research done to resolve this issue [8, 9]. For doing complex data related operations, Stateful Serverless Functions have an obvious need & data handling between the intermediate stages can be a key challenge. Existing works like Cloudburst [10] try to solve the scalability issue, while Faasm [11] proposes a lightweight isolation & Pocket [12] tries to give a heterogeneous storage solution. Similarly, for managing shared state across invocations, AFT [13], HYDROCACHE [14], Crucial [15] offers solutions from different aspects. Ramalingam et al. [16] & Flux [17] discuss idempotence in SSFs. But none of the mentioned works discussed about achieving idempotence by sharing a mutable state, as well as minimalizing the logging overhead as Halfmoon.

Serverless functions should operate as workflows - which aim at providing *exactly-once* semantics, but SSF's make it harder to achieve this goal. Works like Olive [5] address log-based fault tolerance protocol, on top of that Beldi [6] proposes the solution for logging actions during SSF execution, which will be used to prevent duplicates during failures. It motivates Boki [7], by proposing a shared log approach for SSFs. In addition, DDOS [18] tries to tame nondeterminism & reduces logging overhead by only logging messages during arrival times.

The proposed solution, Halfmoon, draws inspiration from all these & enforces determinism by stabilizing timestamps as well as finds a novel approach to eliminate logging overhead.

3 Overview of the Proposed Solution: Halfmoon

To achieve idempotence with minimum logging overhead, the authors propose the solution "Halfmoon", which applies log-based fault tolerance with novel log-optimal protocols. Fig 1 shows the overall structure of the Halfmoon, which decouples the log from the external state into a separate logging layer, much like Boki. The Stateful Serverless Functions (SSF) interact with the external state & the logging layer through Halfmoon's client library. This library comprises of API's that are similar in structure & functionality from existing solutions [6, 7], and does various data operations like read, write & control flow operations. While these APIs have the same signature (function structure) as their raw counterparts, there is a crucial difference.

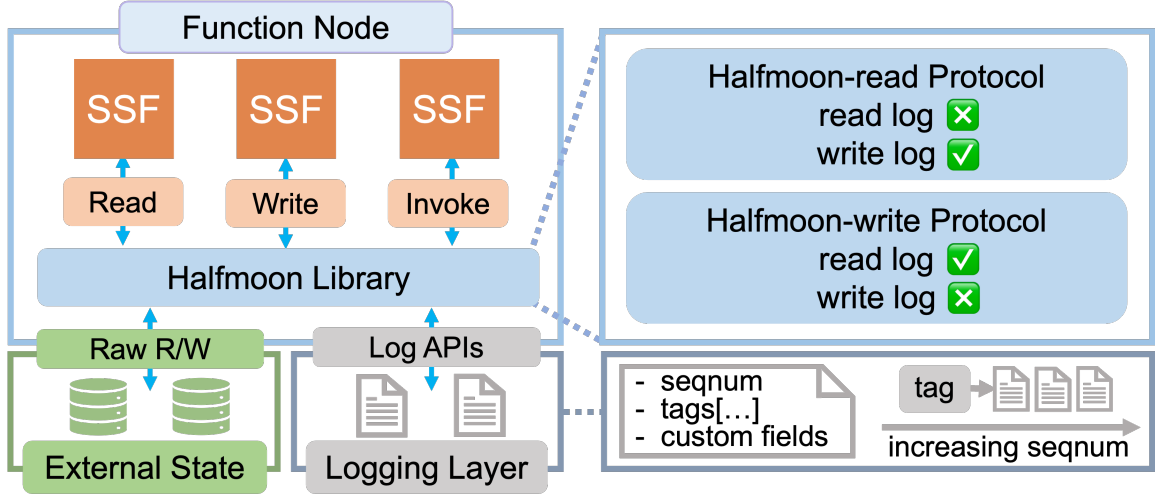


Figure 1: Overall architecture of Halfmoon

The Halfmoon library automatically handles the logging of these operations in the background, which ensures idempotence. It also relies on serverless runtime to detect & re-execute SSFs in failures, which is essential to achieve exactly-one semantics. There are 2 protocols suggested here: Halfmoon-read protocol, which is log-free on reads & Halfmoon-write protocol, which is log-free on writes. The logging layer block in the diagram implements the shared log abstraction, which acts as the even stream of the Halfmoon by enforcing a global overall order of log records. Fig2 lists Halfmoon’s log APIs, all of which have a tag parameter. The idea presented here is that the main log is logically divided into sub-streams containing log records that share a common tag. These records can appear in multiple sub-streams, maintaining a consistent order based on their sequence numbers (seqnums). This organisation reduces the overhead of log replay and allows efficient selective reading, in line with best practices in shared log systems [7, 19, 20].

```

# Return the sequence number of the log record
def logAppend(tags, record) -> seqnum
# Read the previous or next log record
# whose seqnum <= `max_seqnum` or >= `min_seqnum`
def logReadPrev(tag, max_seqnum) -> LogRecord
def logReadNext(tag, min_seqnum) -> LogRecord
# Delete log records up to `seqnum`
def logTrim(tag, seqnum)
# Conditional log append (Section 5.1)
def logCondAppend(tags, record, condTag, condPos)
  -> (seqnum, error)

```

Figure 2: Log APIs in Halfmoon

4 Logging & Timestamp Parameterization

The authors propose to parameterize the reads & writes with timestamps in the event stream & leverage the inherent dependencies among them. Their key insight is, for idempotent reads, the system consistently seeks backwards from the same timestamp. Similarly, for idempotent writes, they should always take effect at the same point in the stream. As an example, in Fig 3:

- F2 executes a read operation at t1 timestamp & sees the latest write from F1 at t0

- While re-executing, instead of seeing F3's write at t3, F2 seeks backward from t1 to t0 in order to recover the previous result - resulting an idempotent read.
- As F2's write has already been once parameterized at t2, even after recovering from the failure it avoids overwriting F3's write - resulting an idempotent write.

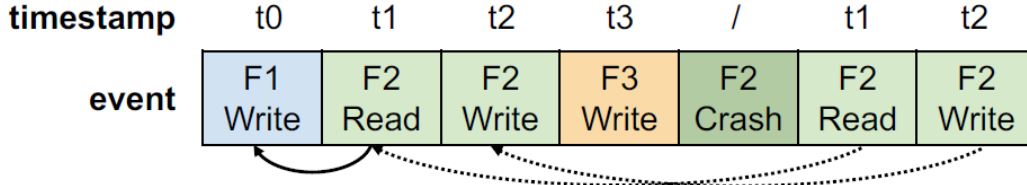


Figure 3: Parameterizing reads and writes with timestamps

Hence, it can be inferred that idempotence boils down to determining the timestamps [18]. But real time events are inherently non-deterministic. In cases where no explicit timestamp is provided (log-free operations), the system needs to calculate timestamps for these events. In addition, these timestamps needs to be bases on the local context of the SSFs, as the order in which SSFs execute & interact with each other can also be non-deterministic.

The logging layer in Halfmoon assigns sequence numbers (seqnums) to log records in a way such that they can also be used as logical timestamps - which eventually ensures a consistent ordering of events. Based on this context, the authors presented "CursorTS" - which is a local & monotonically increasing logical timestamp. It ensures that events are ordered correctly within a particular SSF's context. When an event is logged, the cursorTS is incremented/ advanced, reflecting the order of events. For log-free operations, cursorTS is assigned to provide them with a logical timestamp. This process of obtaining & assigning timestamps follow a deterministic manner, which ensures predictability. An event is either backed by fault-tolerant log records or deterministically inferred from such records. Halfmoon has the capability to reconstruct a complete event stream from partial logs, ensuring that the order of events is maintained even when only some information is available.

5 Halfmoon Design

5.1 Halfmoon Read: Log-Free Read Protocol

The Halfmoon-read protocol is ideal for read intensive workloads. Here, while writes perform the logging, all the reads are log-free - thus minimizing the logging overhead. The Fig. 4, illustrates the lifecycle of a write operation, logging of the new version & subsequent read operation. At first a write operation occurs, creates a new version of the object X with the updated value (X3:4). The logging layer captures this write operation & assigns a logical timestamp (seqnum) with T3 . This monotonically increasing seqnum is handled by the logAppend API. For a read operation to perform, the aforementioned cursorTS(logical timestamp) T2 is used to obtain X2:2 from the external state. This effective order of events follows the order of the seqnum-based logical timestamps. Here X1,X2, X3 writes are performed by the same function on the external state. Logging layer helps to maintain the order & consistency of operations.

Observation - Write Idempotence: The version numbers assigned to these write operations follow a predictable & consistent pattern - hence these are deterministic. It ensures that each time a version of the object is created during a write, is both uniquely identified & sequenced. It's also observed that each log record in the logging layer is also uniquely identified with a sequence number-pointing to a specific write operation which ensures that these log records accurately represent the order & distinctiveness. In summary, these aforementioned properties of deterministic writes, idempotence in object creation & the uniqueness of log records collectively contribute to the idempotence of write operations in Halfmoon.

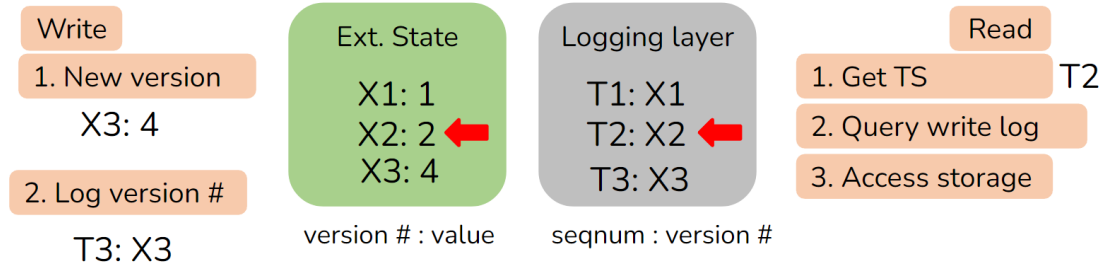


Figure 4: Example of Halfmoon-read protocol

Observation - Read Idempotence: As the log records are unique, the read timestamps & the write log look-ups are also deterministic all the time. That will lead to reading the same version each time - achieving idempotence for the Halfmoon-read protocol.

5.2 Halfmoon Write: Log-Free Write Protocol

Alternatively, Halfmoon-write protocol is applied in write intensive workloads. In Fig.5, the initial records in the external state were Key:X; Value: 2; Version:1. During the write operation, cursorTS is being used & a conditional update occurs for X, the value & version gets updated. For the read operation, consider F3, which wants to read the value of the object X. Following the protocol, it checks the read log, accesses storage for the value (X=2), and logs the read result (T3: F3; X=2). The logging layer saves the log records from different transactions, each of them captures the state of the system at a specific point in time.

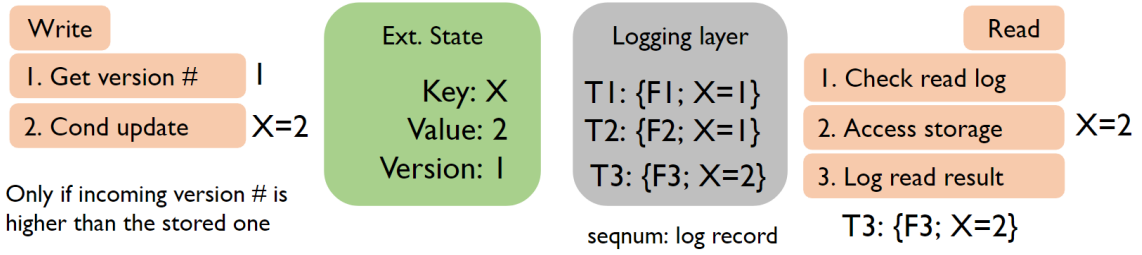


Figure 5: Example of Halfmoon-write protocol

Observation - Write Idempotence: As we are doing conditional updates, they will never overwrite another version, based on the fact that version numbers are deterministic. That eventually leads to the log records being unique as well. Due to this, idempotent writes are achieved.

Observation - Read Idempotence: Reads are directly logging the real-time data observed from the external state, which makes them idempotent on their own.

6 Log Optimality

The authors assume that *under the worst case, a fault tolerant logging protocol either logs all reads or all writes with visible external effects*. This is derived from the hypothesis of having log free writes with visible external effects, which leads to logging all the reads under the worst-case scenario. In addition, Halfmoon measures logging overhead by counting abstract logging operations. During the worst case, it can choose to imply Halfmoon-read or Halfmoon-write based on the workload - always pushing the overhead to a lower bound. This log optimality is theoretically proven by contradictions & lemmas.

7 Evaluation

Halfmoon’s performance is compared with Boki [7], using microbenchmarks & real-time application loads. Also, under different read/write intensities, Halfmoon’s storage and runtime overhead is calculated. Lastly, the switching delay between the protocols has been discussed.

Experimental Setup: All the experiments are conducted in AWS EC2 instances (c5d.2xlarge VMs, with 8 vCPUs, 16GiB DRAM, 200GB SSD) using Boki as the serverless runtime [7]. Amazon DynamoDB [21] is used as the external storage for both Boki & Halfmoon.

7.1 Microbenchmarks

To orchestrate this, the external state was populated with 10K objects. In Fig.6, it can be observed that Halfmoon offers 30% lower latency compared to Boki on reads. achieves similar performance in writes. It also offers exactly-once read with 15% overhead compared to raw reads, which is still lower than Boki. As Halfmoon performs expensive *conditional* updates, the latency of log-free writes is still higher than raw writes. Here in the figure, the main bars & error bars show the median and 99%-tile tail.

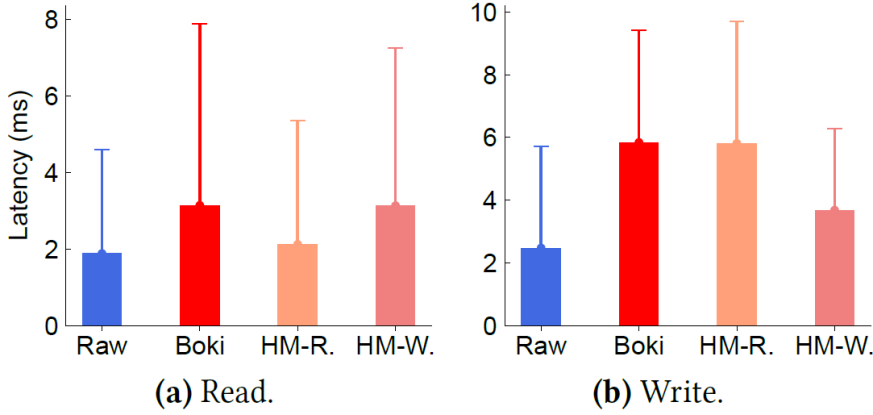


Figure 6: Read/Write latency between Boki & Halfmoon

7.2 Application Workloads

Halfmoon is evaluated with three realistic application workloads. The travel reservation & the movie review system is adapted from DeathStarBench [22, 23], and used in both Boki & Beldi. The travel reservation is a read-intensive operation, that runs a workflow of 10 SSFs. The movie review application is by nature write-intensive (13 SSFs), making it ideal for Halfmoon-write testing. Retwis, a simplified Twitter clone [24] is also read-intensive. All of them store data in DynamoDB. Authors evaluated both Halfmoon-read & Halfmoon-write to demonstrate the benefits of using the appropriate protocol, as well as worst-case performance penalties for using the wrong one. In Fig.7, it’s observed that Halfmoon offers 20%-40% lower end-to-end latency using appropriate protocols, with 1.5-4.0x lower logging overhead above the unsafe baseline. It’s also noteworthy to mention that even after using the wrong protocol, Halfmoon outperforms Boki.

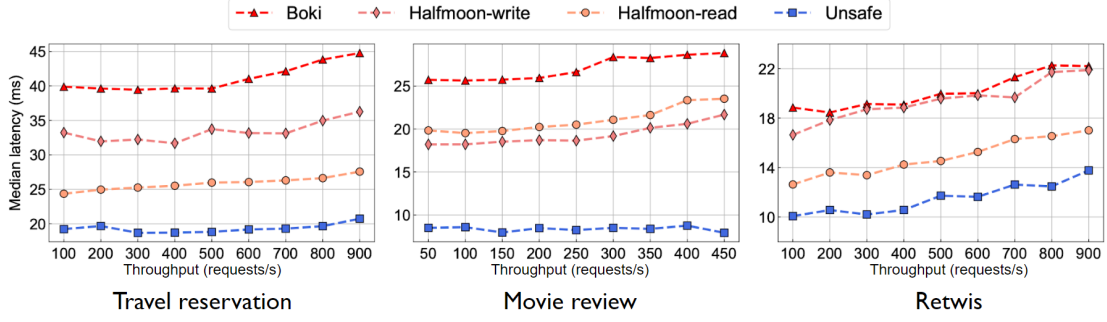


Figure 7: End-to-end performance of Boki & Halfmoon under three realistic application workloads

7.3 Switching Delay

Using a two-phase workload, the switching delay is being evaluated between the two protocols. The first phase runs Halfmoon-write with a read ratio of 0.2, changes the phase every 5 seconds to run Halfmoon-read with a read ratio of 0.2. In Fig. 8, under a moderate workload (a), the switching takes under 100ms. with a higher workload (b), it takes longer to switch from Halfmoon-write to Halfmoon-read. This happens because due to write write intensity in the first phase, Halfmoon needs to wait until the old protocol is completed before switching to the new one.

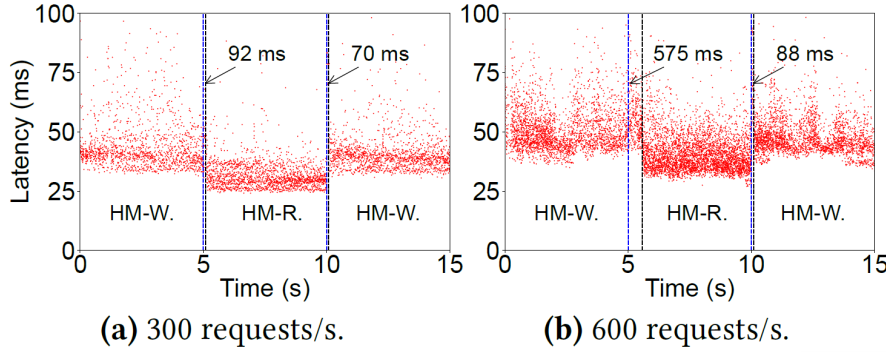


Figure 8: Switching delay between Halfmoon protocols

8 Personal Insights

After thoroughly reading about the problem that the authors tried to address along with the proposed solutions in the paper, I had some key observations:

Dependency on Theoretical Assumptions: It's safe to say that the author's assumptions, definitions & lemmas result in valuable insights about further optimization based on log optimality. But while reading the paper, it sometimes felt that the approaches are too much theoretical. For example, proof by contraction of log-free writes & extending that into proving the worst case scenario. The approach to the proof - always pushing the log-overhead to lower bound seemed too theoretical as well. Also, from time to time under the *Evaluation* section, a lot of approaches and results have been derived based on these prepositions.

Insensitivity to Garbage Collection Interval: The fact that Halfmoon's runtime performance isn't much affected by the GC interval is very fascinating to me. From a developer's point of view, it might be helpful in different deployment environments.

Multi Versioning: In Halfmoon-read, multi-versioning impacts the storage usage. I think this might be helpful while designing systems.

Switching Delay: Despite stating in the earlier sections of the paper that the switching between the protocols is seamless & pauseless, later it was revealed that it depends on very particular factors i.e. low to moderate workloads, which protocol comes first etc. As Halfmoon's boundary conditions

are based on read/write intensity and choosing the right protocol based on that, this switching can be a crucial factor in massive, complex workloads.

Overall, I think Halfmoon competes effectively with the state-of-the-art solution Boki in terms of runtime and storage efficiency.

9 Conclusion

In conclusion, Halfmoon proposes a novel approach for log-based fault tolerance in SSFs by reducing logging overhead. The two protocols provide either log-free reads or writes, based on the workload - hence halving the overhead. This asymmetrical logging also enforces exactly-once semantics while maintaining idempotence. It's theoretically proven that the protocols are log optimal and the proposed solution always pushes the log overhead to its lower bound. It's suitable for dynamic workloads because of its pauseless switching mechanism. It's also observed that Halfmoon overperforms the state-of-the-art solution in several experiments.

References

- [1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tressness, M. Russinovich, R. Bianchini, Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider, in: 2020 USENIX annual technical conference (USENIX ATC 20), 2020, pp. 205–218.
- [2] G. McGrath, P. R. Brenner, Serverless computing: Design, implementation, and performance, in: 2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW), IEEE, 2017, pp. 405–410.
- [3] T. Lynn, P. Rosati, A. Lejeune, V. Emeakaroha, A preliminary review of enterprise serverless cloud computing (function-as-a-service) platforms, in: 2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), IEEE, 2017, pp. 162–169.
- [4] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, C. S. Meiklejohn, Durable functions: Semantics for stateful serverless, Proceedings of the ACM on Programming Languages 5 (OOPSLA) (2021) 1–27.
- [5] S. Setty, C. Su, J. R. Lorch, L. Zhou, H. Chen, P. Patel, J. Ren, Realizing the {Fault-Tolerance} promise of cloud storage using locks with intent, in: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 2016, pp. 501–516.
- [6] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, V. Liu, Fault-tolerant and transactional stateful serverless workflows, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), 2020, pp. 1187–1204.
- [7] Z. Jia, E. Witchel, Boki: Stateful serverless computing with shared logs, in: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, 2021, pp. 691–707.
- [8] J. M. Hellerstein, J. Faleiro, J. E. Gonzalez, J. Schleier-Smith, V. Sreekanti, A. Tumanov, C. Wu, Serverless computing: One step forward, two steps back, arXiv preprint arXiv:1812.03651 (2018).
- [9] J. Schleier-Smith, V. Sreekanti, A. Khandelwal, J. Carreira, N. J. Yadwadkar, R. A. Popa, J. E. Gonzalez, I. Stoica, D. A. Patterson, What serverless computing is and should become: The next phase of cloud computing, Communications of the ACM 64 (5) (2021) 76–84.
- [10] V. Sreekanti, C. Wu, X. C. Lin, J. Schleier-Smith, J. M. Faleiro, J. E. Gonzalez, J. M. Hellerstein, A. Tumanov, Cloudburst: Stateful functions-as-a-service, arXiv preprint arXiv:2001.04592 (2020).
- [11] S. Shillaker, P. Pietzuch, Faasm: Lightweight isolation for efficient stateful serverless computing, in: 2020 USENIX Annual Technical Conference (USENIX ATC 20), 2020, pp. 419–433.

- [12] A. Klimovic, Y. Wang, P. Stuedi, A. Trivedi, J. Pfefferle, C. Kozyrakis, Pocket: Elastic ephemeral storage for serverless analytics, in: 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), 2018, pp. 427–444.
- [13] V. Sreekanti, C. Wu, S. Chhatrapati, J. E. Gonzalez, J. M. Hellerstein, J. M. Faleiro, A fault-tolerance shim for serverless computing, in: Proceedings of the Fifteenth European Conference on Computer Systems, 2020, pp. 1–15.
- [14] C. Wu, V. Sreekanti, J. M. Hellerstein, Transactional causal consistency for serverless computing, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 2020, pp. 83–97.
- [15] D. Barcelona-Pons, P. Sutra, M. Sánchez-Artigas, G. París, P. García-López, Stateful serverless computing with crucial, ACM Transactions on Software Engineering and Methodology (TOSEM) 31 (3) (2022) 1–38.
- [16] G. Ramalingam, K. Vaswani, Fault tolerance via idempotence, in: Proceedings of the 40th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages, 2013, pp. 249–262.
- [17] H. Ding, Z. Wang, Z. Shen, R. Chen, H. Chen, Automated verification of idempotence for stateful serverless applications, in: 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23), 2023, pp. 887–910.
- [18] N. Hunt, T. Bergan, L. Ceze, S. D. Gribble, Ddos: taming nondeterminism in distributed systems, ACM SIGPLAN Notices 48 (4) (2013) 499–508.
- [19] M. Balakrishnan, D. Malkhi, T. Wobber, M. Wu, V. Prabhakaran, M. Wei, J. D. Davis, S. Rao, T. Zou, A. Zuck, Tango: Distributed data structures over a shared log, in: Proceedings of the twenty-fourth ACM symposium on operating systems principles, 2013, pp. 325–340.
- [20] M. Wei, A. Tai, C. J. Rossbach, I. Abraham, M. Munshed, M. Dhawan, J. Stabile, U. Wieder, S. Fritchie, S. Swanson, et al., {vCorfu}: A {Cloud-Scale} object store on a shared log, in: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17), 2017, pp. 35–49.
- [21] M. Elhemali, N. Gallagher, B. Tang, N. Gordon, H. Huang, H. Chen, J. Idziorek, M. Wang, R. Krog, Z. Zhu, et al., Amazon {DynamoDB}: A scalable, predictably performant, and fully managed {NoSQL} database service, in: 2022 USENIX Annual Technical Conference (USENIX ATC 22), 2022, pp. 1037–1048.
- [22] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, et al., An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems, in: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, 2019, pp. 3–18.
- [23] Deathstarbench, <https://github.com/delimitrou/DeathStarBench/>, accessed: 29 January 2024.
- [24] Tutorial: Design and implementation of a simple twitter clone using php and the redis key-value store., <https://redis.io/docs/manual/patterns/twitter-clone/>, accessed: 29 January 2024.